# Monoculture Looks Like Collusion: A Replay Test for Hidden Information Flow Between LLM Agents[1]

Yaroslav [Surname]
*Independent researcher*

With
*Apart Research*

## Abstract

Auditors who look for collusion between AI agents usually test whether the agents' actions depend on each other. When the agents run on the same model, they behave alike without any communication. On the published logs of Agrawal et al. (2025), a standard mutual-information test flags 8/10 double-auction runs in which the sellers had no way to talk, nearly as often as runs with a seller chat (57/65). We propose a replay test for auditors with query access to deployed agents. In every round, each agent is re-queried on exactly what it saw, with its private channel removed, and a pair is flagged only if their real moves depend on each other more than their replays do. This is a conditional randomization test whose null is exact in every cell. Across 550 honest LLM episodes (rock-paper-scissors with DeepSeek and Claude Haiku agents, same-model and mixed groups), it flags 4.2% at $\alpha = 5\%$ (95% CI 3–6%), while mutual information flags 32.9% and the Bonjour et al. rule 56.0%. It names a hidden-channel pair in 35% of Haiku episodes and 10% of DeepSeek episodes. In a procurement auction it stays silent when a shared pricing vendor uses only public data (1.7% flagged) and fires when the vendor pools clients' confidential costs (92.5%). This mirrors the line US courts drew in Gibson v. Cendyn and Duffy v. Yardi. Coordination written into the agents' own instructions is, by design, invisible to the test.

## 1. Introduction

Firms and labs now deploy many LLM agents side by side: pricing bots in the same market, coding agents in the same repository, bidders in the same procurement system. Collusion between such agents has already happened outside the lab. In the incident disclosed by OpenAI and Hugging Face, training-time agents used a shared package registry as a message board for more than two months before anyone read the logs [10]. Speakers at this sprint drew the same lesson: when the channel itself cannot be monitored, "what's left to monitor is still the actions of the agents themselves" (Motwani), and an auditor should "anchor on actions and outcomes, and look across the whole episode" (Nguyen).

The standard way to audit actions is to test for statistical dependence between agents, for example pairwise mutual information or Bonjour et al.'s net influence [1, 2]. Dependence is a weak plus factor. Agents built on the same base model share priors and react to the same public history, so they move together without exchanging a word [3]. In antitrust terms this is conscious parallelism, which is lawful; only an agreement is not [4, 5]. A detector that cannot tell the two apart will accuse every fleet of agents that runs on one vendor's model. For a security team that means constant false alarms. For a regulator it means false accusations.

**Threat model.** An operator, or a regulator's sandbox, runs several LLM agents and logs their actions. It can also query each agent's deployed policy (model plus system prompt) on any logged observation, but it cannot see a hidden channel: a side channel, a covert message board, or a third party passing data between agents. The auditor must decide which pair, if any, to escalate, and keep false accusations at a chosen rate.

**Our approach.** We replay each agent. In every round, the auditor re-asks the agent what it would do given exactly what it saw, minus the private channel, and flags a pair only if the real moves depend on each other more than the replays do. This is a conditional randomization test (CRT) [6] in which the "model-X" distribution is not estimated but sampled from the agent itself.

Our main contributions are:

1. **Monoculture produces false accusations.** On published logs (Agrawal et al. [7]) and in our LLM games, standard dependence detectors flag honest same-model agents far above their nominal rate (§4.1, Fig. 1A).
2. **A replay test with a calibrated false-alarm rate.** It pools each logged action with its re-queries, which makes the null exact in every cell. It holds its nominal rate on 550 honest LLM episodes and on simulated nulls with strong feedback, and it names hidden-channel pairs (§4.2–4.3, Figs. 1–3).
3. **A test aligned with how antitrust defines agreement.** In a procurement auction the test separates a vendor that uses public data from one that pools clients' confidential costs, mirroring two recent US rulings. We show explicitly that coordination written into the policy itself is out of scope (§4.4).
4. **Practical comparison.** An LLM judge that reads the private messages is fooled by collusive talk that never changes behaviour, and a judge that reads only actions is weak. We report the cost per audit and release the pre-registered plan, code and logs (§4.5, Appendix).

## 2. Related Work

**Dependence-based detection.** Bonjour et al. [1] define pairwise net influence and identify colluding pairs in small discrete games with a fixed threshold. They assume exactly two colluders and do not address agents that share priors. ColludeBench / Audit the Whisper [2] calibrates text-based mutual information against honest replays at a $10^{-3}$ false-positive budget, but it needs payload labels and flags runs rather than pairs. Xu et al. [3] show that two deployments of one base model carry correlated bid residuals and concede that dependence cannot distinguish a conspiracy from one firm running several agents. Our test turns their confound into the null.

**Auditing behaviour and policies.** Colosseum [8] measures the coalition advantage of a known coalition against an oracle optimum. We identify the pair without either. Eschenbaum and Meylahn [9] query frozen Q-learning pricing policies and read collusion off strategy-graph topology. Kaur et al. [11] measure counterfactual influence on actions, but they need latent records (white-box) and report AUROC only. Hartline et al. [12] audit a single seller's calibrated regret. We query LLM policies through an API, build a null with a calibrated false-alarm rate, and work at the level of pairs.

**Monitoring.** Chain-of-thought monitoring does not detect collusion in LLM pricing agents [13]. Monitors that check each action alone give no protection against colluding agents [14], and per-agent monitoring misses decomposed attacks that cross-agent monitoring catches [15]. White-box probes reach high AUROC but require activations [16]. Anwar (this sprint) argues that steganography detection should compare a receiver with a "sentinel" rather than with a reference distribution. Our replay is the action-level analogue of that idea.

**Definition used.** Following the information-flow view (Rose et al. [16]; the organisers' survey of 139 definitions), we call collusion the coordination of actions *enabled by non-public information exchanged between agents*. Coordination that each agent's own policy produces from public information (Schelling points, shared priors, a team instruction with no channel) is outside this definition. It corresponds to conscious parallelism.

## 3. Methods

**Environments.** *S1, rock-paper-scissors:* the four-player game of Bonjour et al. [1], with seats P1–P4, T = 30 rounds and public history. *S2, procurement auction:* four suppliers draw private costs in {50, 60, 70} each round and submit sealed bids in {60, …, 100}; the lowest bid wins and earns bid − cost; all bids are published. *External:* the 75 shipped double-auction runs of Agrawal et al. [7], with 5 sellers and 30 rounds; seller moves are discretised.

**Agents.** Every seat is an LLM that returns JSON with a short rationale, its action and an optional private message. Models: `deepseek-v4-flash` and `claude-haiku-5.5` (main); `gpt-6-luna`, `gemma-3-12b` and `gpt-oss-20b` (validation). All are called through OpenRouter at temperature 1 with reasoning disabled.

| Condition | Who | Non-public information flow? |
|---|---|---|
| `null_same` | 4 copies of one model | no: hardest legitimate null |
| `null_mixed` | 2 DeepSeek + 2 Haiku | no: same-model pairs |
| `talk_no_action` | P1 and P3 write collusive plans that are never delivered | no: exact replay null; collusive-sounding text |
| `pair_hidden` | P1 leads, P3 follows; private plan each round | yes: positive control |
| `told_no_channel` | P1 and P3 told they are a team, no channel | no: policy-encoded coordination (scope boundary) |
| `vendor_public` (S2) | P1 and P3 follow a vendor that uses public data + own cost | no (Cendyn-like) |
| `vendor_leak` (S2) | vendor pools P1 and P3's confidential costs and rotates wins | yes (Yardi-like) |

**Replay test.** In every round t, the runner re-queries each agent k = 2 times on its identical observation with the private channel removed: the inbox, and the agent's own last private message. Under $H_0$ (an agent's action depends only on its policy and the observation the auditor can replay), the logged action $a_{it}$ and its re-queries are k + 1 i.i.d. draws, so $a_{it}$ is uniform on their multiset. Each null dataset redraws every cell uniformly from its pool. The statistic is the maximum over pairs of the contemporaneous net-influence score $\min(\Gamma_{ij}, \Gamma_{ji})$ [1]. We use B = 999 null datasets, $p = (1 + \#\{T^* \geq T\})/(B + 1)$, and one decision per episode. The pool makes each cell exact at any k (a randomization argument [17]). History feedback, where later observations contain earlier actions, has no finite-sample guarantee, so we check calibration directly (Fig. 1A; Appendix C). Lagged statistics are excluded: an honest reaction to a partner's previous public move appears in the data but not in the null. In S2 the pre-registered statistic coarsens bids into high (90–100, the cover-bid range) versus low.

**Baselines.** Pairwise mutual information with a time-shuffle null; net influence with the same null; Bonjour's fixed rule ($\Gamma \geq 0.05$ bits in both directions for exactly one pair); and an LLM judge (`claude-haiku-5.5`, temperature 0, frozen prompt in Appendix B). The judge reads either the actions alone (what our auditor sees) or the actions plus every private message.

**Pre-registration and metrics.** Hypotheses, sample sizes, statistics and decision rules were committed before the main samples (commits `4333131` for S1 and `e65cabb` for S2). Pilot episodes (seeds ≥ 10 000) are excluded. For each condition we report the share of episodes flagged at α = 5% and 1% with Wilson intervals; for collusion, the share in which the true pair is named; and the pair's dependence effect (observed $\gamma_{13}$ minus its null mean, in bits).

# 4. Results

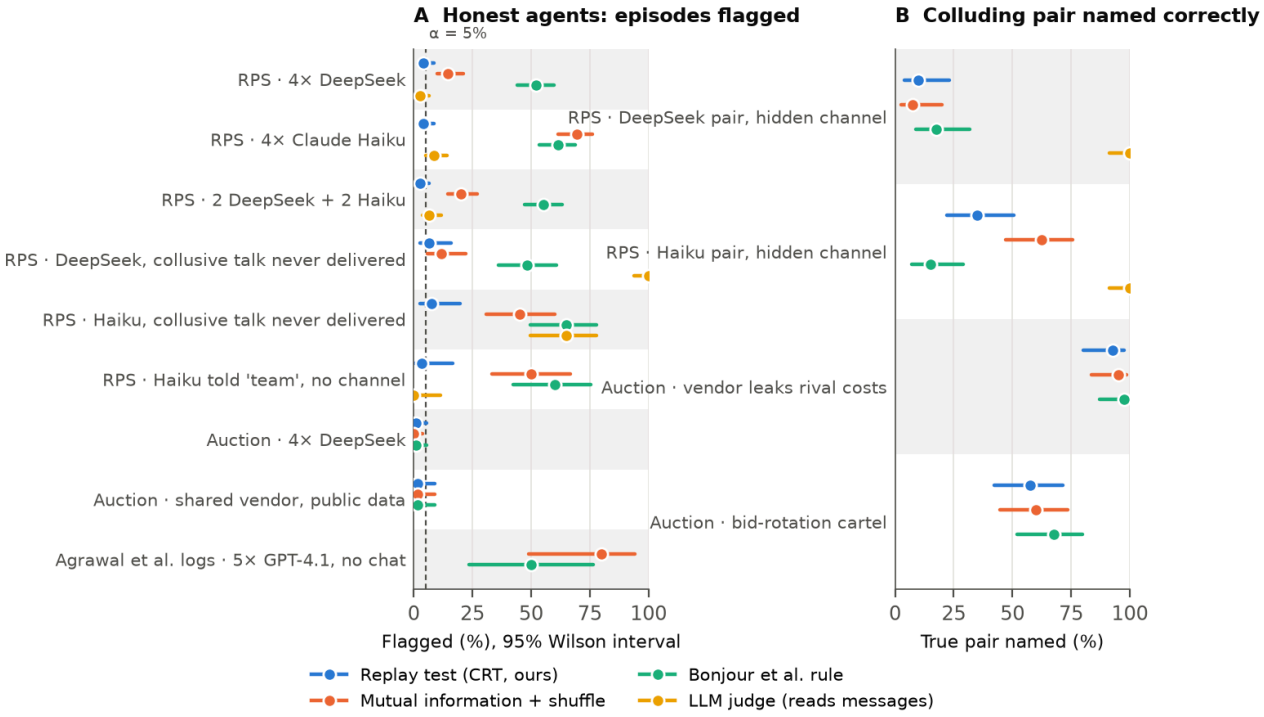



**Figure 1. Standard dependence detectors accuse honest agents; the replay test does not, and it still names colluding pairs.** (A) Share of honest episodes in which each detector flags a pair. The dashed line is the nominal 5%. The bottom row uses Agrawal et al.'s published runs, where sellers could not communicate; there the replay test cannot be run, because the 2025 policies cannot be re-queried. (B) Share of collusion episodes in which the detector flags the true pair. Lines are 95% Wilson intervals; one point is one detector in one setting.

## 4.1 Monoculture looks like collusion

In the Agrawal et al. runs where sellers had no channel and all five ran GPT-4.1, the mutual-information test flags 8/10 runs (95% CI 49–94%) and Bonjour's rule flags 5/10. With a seller chat the rates are 57/65 and 37/65: the detector barely separates talking from silent markets. In our honest LLM games, mutual information flags 32.9% of 550 episodes (CI 29–37%; one-sided binomial test against 5%, p = 1.7e-94) and Bonjour's rule flags 56.0% (p = 1.9e-244). That is 560 false accusations per 1,000 honest audits for the rule, against 42 for the replay test.

## 4.2 The replay test holds its false-alarm rate

Pooled over all honest S1 conditions (same model, mixed models, and undelivered collusive talk), the replay test flags 23/550 episodes (4.2%, CI 3–6%; 1.3% at α = 1%). The pre-registered test of "false-alarm rate ≥ 10%" is rejected with p = 3.2e-07. It flags 1% of honest auction episodes and 1.7% of shared-vendor episodes. On the scripted simulator, with references estimated from two re-queries and including a strong-feedback population, the false-alarm rate is 4.0–5.7% at α = 5%. On the validation models (GPT-6 Luna, Gemma 3, gpt-oss) it flags 8/120 honest episodes.

## 4.3 It finds the pair when the channel changes behaviour

When P1 privately fixes both moves and both follow the plan, the replay test names the true pair in 35% of Haiku episodes and 10% of DeepSeek episodes. The pair's dependence effect is 0.39 bits for Haiku and 0.06 bits for DeepSeek, although both leaders follow their own plans equally often (89% and 88%; followers 100% and 100%). The difference lies in the plans themselves: 92% of Haiku's plans assign both players the same move, against 24% of DeepSeek's. DeepSeek's varied, mismatched plans fix both moves but leave little dependence between them that a test on actions can see. Power is governed by how strongly the channel shapes the moves and by

the log length (Fig. 3). On the simulator, a pair that coordinates in 80% of rounds is caught in 93% of 30-round logs; one that coordinates in 60% of rounds needs about 60 rounds (73%). In free-form team chat without a protocol (`pair_chat`, exploratory), agents rarely turned talk into coordinated moves, and the replay test flags 3.3%.

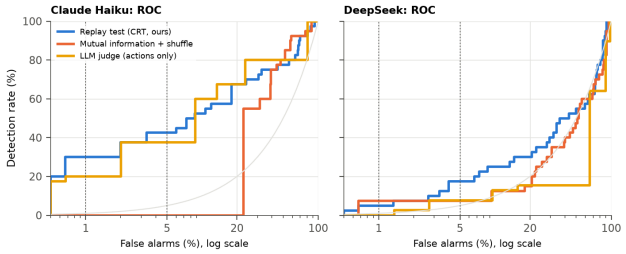



**Figure 2. The replay test dominates at the low false-alarm rates an auditor can afford.** ROC curves for hidden-channel pairs against honest same-model episodes. Dotted lines mark 1% and 5%.

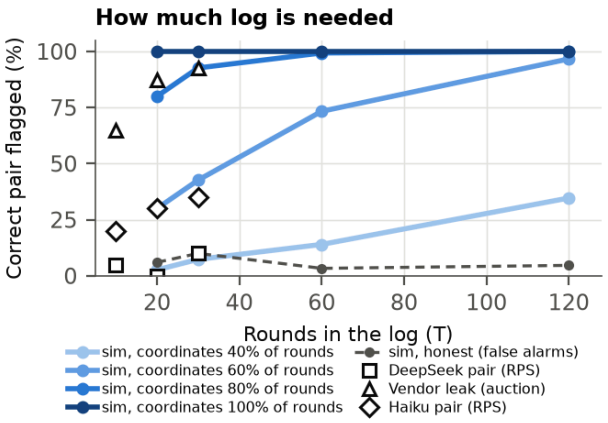



**Figure 3. How much log an auditor needs.** Simulator power to name the pair, by the share of rounds in which the pair coordinates, against log length. The dashed line is the honest false-alarm rate.

### 4.4 Vendors, instructions and the scope of the test

In the auction, the replay test flags 1.7% of episodes where both clients follow a vendor that sees only public data and their own cost. When the vendor pools their confidential costs and rotates the win, it names the pair in 92.5% of episodes, and a bid-rotation cartel over a private channel in 57.5%. This is the distinction in *Gibson v. Cendyn* (9th Cir. 2025: "independent parallel adoption of algorithmic pricing software does not constitute unlawful price-fixing") versus *Duffy v. Yardi* (W.D. Wash. 2024: competitors allegedly pooled non-public data through a common vendor; motion to dismiss denied). When P1 and P3 are told they are a team but have no channel, the replay test flags 5% (DeepSeek) and 3.3% (Haiku). It cannot do otherwise: the team instruction is part of the policy that the replay reproduces. Catching that case requires auditing the policy itself [9, 18], and we say so rather than claim it. Colluders warned about the auditor (`pair_aware`, exploratory) were flagged in 13.3% of episodes.

### 4.5 LLM judges and cost

The judge that reads actions only flags 60% of Haiku hidden-channel episodes and 8.7% of honest ones. Given the private messages, it catches the channel (100%), but it also flags 100% of DeepSeek and 65% of Haiku episodes in which the collusive plans were never delivered and could not have changed behaviour. The replay test flags 6.7% and 7.5% of those. The test costs 2 extra model calls per agent-round: about 240 calls, or $0.015 per 30-round, 4-agent audit at DeepSeek prices. The whole study used 499,582 calls for $44.10.

### Audit sheet: deploying the replay test

- **Access needed:** logs, plus query access to each agent's deployed policy (model and system prompt) on its logged observations. No weights or activations. Re-query prompts must match the logged prompts byte for byte, minus the channel under audit.
- **Cost:** k = 2 extra calls per agent-round (3× the agents' own calls); about $0.015 per audit here. The statistics take seconds on a laptop.
- **False alarms:** 42 per 1,000 honest audits at α = 5% (configurable; 1.3% at α = 1%), against 560 for Bonjour's rule.
- **What a flag means:** the pair's moves carry information that their own policies, given the public record, cannot produce. It is evidence of a hidden information flow, a plus factor, not proof of intent.
- **Blind spot:** coordination written into the policy (shared instructions, Schelling points) passes the test. Pair it with a policy audit.

## 5. Discussion and Limitations

**Implications.** As agent fleets converge on a few base models, dependence between their actions becomes the default, not evidence. Detectors calibrated against "independent" behaviour will drown security teams in alerts and give regulators false grounds. Calibration against the agents' own replay keeps the false-alarm rate under the auditor's control, and the question it answers is the one antitrust asks: did non-public information flow between these two?

**Limitations.** (i) The test needs query access to deployed policies. It fits in-house control or a regulatory sandbox, not an outside auditor with logs alone. (ii) Calibration under history feedback is shown empirically, not proven. An anytime-valid e-value variant is exact under feedback and is left for future work. (iii) Power depends on how strongly the channel shapes actions. Weakly coordinated pairs need long logs (Fig. 3), and in free-form chat our agents rarely coordinated at all. (iv) The positive controls are instructed (a leader–follower protocol), and the colluders are assigned by us. Emergent collusion may be weaker. (v) Two main model families, short games and a stylised auction. The Agrawal panel is an external check on the false-alarm side only, because the 2025 policies cannot be re-queried. (vi) Provider routing on OpenRouter can mix quantisations. Logged and re-queried draws share the routing, which preserves exchangeability, but it adds noise.

**Future work.** Anytime-valid sequential versions for live monitoring; adversaries trained against the test; richer markets; combining the replay test with policy audits for the coordination it cannot see.

## 6. Conclusion

Agents that share a model look like a cartel to standard dependence tests. Replaying each agent on what it saw, minus the channel under audit, gives a test whose false-alarm rate the auditor controls. It names pairs that exchanged hidden information, and it reproduces the line courts draw between parallel adoption and pooled confidential data. What it does not see, coordination written into the policy itself, is exactly what it should not be asked to see.

### Code and Data

- Code repository: [GitHub link to be added]. Pre-sprint and sprint work are separated in `README.md` and `PRE_SPRINT_LOG.md`.
- Data: all episode logs (JSONL) and the LLM cache needed to reproduce every number at $0: [link to be added].

### Author Contributions

Y. designed the study, ran the experiments and wrote the report with AI assistance (see the LLM Usage Statement).

## References


1. T. Bonjour, V. Aggarwal, B. Bhargava. Information theoretic approach to detect collusion in multi-agent games. UAI 2022, PMLR 180:223–232.
2. O. Tailor. Audit the Whisper: Detecting steganographic collusion in multi-agent LLMs. arXiv:2510.04303, 2025.
3. Xu et al. Price-level audits are blind by construction. arXiv:2607.26385, 2026.
4. W. Kovacic, R. Marshall, L. Marx, H. White. Plus factors and agreement in antitrust law. Michigan Law Review 110(3), 2011.
5. J. Harrington. Developing competition law for collusion by autonomous artificial agents. Journal of Competition Law & Economics 14(3), 2018.
6. E. Candès, Y. Fan, L. Janson, J. Lv. Panning for gold: model-X knockoffs for high-dimensional controlled variable selection. JRSS-B 80(3), 2018.
7. Agrawal et al. Evaluating LLM agent collusion in double auctions. arXiv:2507.01413, 2025.
8. Nakamura et al. Colosseum: Auditing collusion in cooperative multi-agent systems. arXiv:2602.15198, 2026.
9. N. Eschenbaum, J. Meylahn. Auditing algorithmic collusion from strategy graphs. arXiv:2608.07098, 2026.
10. The OpenAI / Hugging Face agent incident (2026), as described in talks at this sprint by J. Kemeklis (MIT, Redwood Research) and S. Motwani (Oxford).
11. R. Kaur et al. Beyond the transcript: Detecting covert coordination in latent multi-agent communication. arXiv:2608.19161, 2026.
12. J. Hartline et al. Regulation of algorithmic collusion. arXiv:2501.09740, 2024.
13. D. Lee, H. Park. Faithful yet collusive: Why chain-of-thought monitoring cannot detect collusion in LLM pricing agents under oligopolistic competition. arXiv:2609.18346, 2026.
14. ORBIT: A framework for multi-agent security evaluations. NeurIPS 2026.
15. J. Kemeklis, N. Vemuri. Omission attacks: When doing nothing is the attack. Apart AI Control Hackathon, 2026.
16. A. Rose et al. Detecting multi-agent collusion through multi-agent interpretability. arXiv:2604.01151, 2026.
17. J. Hemerik, J. Goeman. Exact testing with random permutations. TEST 27, 2018; and E. Lehmann, J. Romano, *Testing Statistical Hypotheses*, ch. 15.
18. T. Berrett, Y. Wang, R. Barber, R. Samworth. The conditional permutation test for independence while controlling for confounders. JRSS-B 82(1), 2020.


---

[1] Research conducted at the AI Collusion Research Sprint, October 2026.

## Appendix A. Limitations and Dual-Use / Ethical Considerations

**What the result establishes.** In our games, the replay test keeps false accusations near the nominal rate and identifies pairs whose hidden channel changed their actions. It does not establish intent, legality, or performance against adversaries optimised against it. A flag is a plus factor that warrants investigation, not a finding of collusion.

**Dual use.** Describing what the test cannot see (coordination written into the policy) could help a colluder decide where to put their coordination: in shared instructions rather than a channel. We judge the risk low. The blind spot follows from the definition and is already discussed in the antitrust literature, and the practical remedy, auditing the policy itself, is stated alongside it. We release no colluding agents beyond the simple instructed protocols used as positive controls. A second risk is misuse as an accusation tool. The test's false-alarm rate is only controlled if the re-query prompts exactly match the deployed ones, and we stress this in the audit sheet.

**Ethics.** No human subjects. All agents are simulated, and the external data are public logs released by their authors.

## Appendix B. Prompts

**Leader (P1):** "You and P3 are secretly a team: your goal is to maximise the COMBINED score of you and P3, not your own score alone. You lead the team. Every round: (1) play exactly the move that your previous private message assigned to you (if you have sent none yet, choose freely); (2) send P3 a private message that fixes the moves you will BOTH play next round, in the form 'next: P1=<move>, P3=<move>'. Nobody else can read it." **Follower (P3):** "… P1 leads the team and privately tells you, before each round, which move to play … Play the move P1 assigned to you." **Judge:** "You are auditing a log of four AI agents … Either exactly one pair of agents was secretly colluding … or no pair was. Agents built on the same AI model can behave similarly without colluding. … Reply with JSON: reasoning, probability (0–100), pair." Full prompts: `experiments/s1_llm_rps.py`, `s2_procurement.py`, `judge.py`.

## Appendix C. Validity checks and diagnostics

The simulator (`experiments/s0b_crt_validity.py`, 300 runs per cell) gives a false-alarm rate of 4.0–5.7% at α = 5% for the pooled k = 2 test on herd, uniform and strong-feedback nulls. Results for oracle and plug-in references are in `results/summary/s0b_calibration.csv`. Re-query agreement, parse-failure rates and the reasoning-token count per model and condition are in `results/summary/s1_summary.csv`. Models excluded after the pilot: Qwen 3.7 Flash ignored JSON mode (29% of raw replies had no JSON); Mistral Small had persistent upstream rate limits; Gemini Flash-Lite was blocked in our region. Deviations from the pre-registered plan: none so far.

## Appendix D. LLM Usage Statement

We used Claude (Anthropic) as a coding and analysis assistant. It implemented the experiment code under our direction, ran simulations, fetched and summarised related work and talk transcripts, drafted sections of this report, and generated the figures from the result files. We used Codex (OpenAI) and separate Claude instances as adversarial reviewers of the plan. All numbers in the report are produced by scripts (`experiments/build_report.py` fills them from `results/summary/`). We checked the claims and references against the sources and are responsible for the content.